

Section 7: AI Engineering Interview

This section covers the AI-engineering substrate of Helios: the Model Context Protocol that turns trust principles into wiring, the tool-calling contract that makes governed SQL and deterministic math enforceable rather than aspirational, and the multi-agent orchestration that runs the diagnosis loop on a deterministic state machine. The throughline is a single thesis — the hard part of an AI analyst is not generating insights, it is making them correct and trusted — and every answer ties an industry-general concept back to a specific Helios server, agent, or guardrail.

MCP

Q1. What is the Model Context Protocol, and what problem does it actually solve?

Ideal answer. MCP is an open client-server protocol that lets an LLM-driven agent invoke external capabilities ("tools"), read external state ("resources"), and reuse parameterized instruction templates ("prompts") over a single schema-typed JSON-RPC 2.0 interface. A server advertises a manifest of tools, each with a JSON Schema for its inputs and outputs; the client (the model runtime) marshals the model's structured tool calls to the server and returns typed results into context. The problem it solves is integration sprawl and trust: instead of every agent hand-rolling bespoke glue to each system, you get a uniform, introspectable, versionable capability surface — and because the surface is the *only* way the model touches the outside world, you can make that surface a hard boundary. The win is not "the model can call functions"; the win is that capabilities become declarative, discoverable, and governable. **Why Helios demonstrates it.** Helios exposes exactly five MCP servers (warehouse, semantic, stats, experiment, report) and the Claude Agent SDK acts as the MCP client, loading them from `mcp_servers.yaml`, fetching each manifest, and exposing the union of tools to the agents — so every database access and every number is a typed tool call, never free text. **Follow-ups.** How does a JSON-RPC tool manifest differ from an OpenAPI spec for the same backend? / What does MCP give you that a hand-written tool registry inside one process does not? / How do resources and prompts differ from tools, and did you use them in Helios?

Q2. Why adopt a tool *protocol* at all instead of just importing Python functions the agent can call?

Ideal answer. A protocol buys you process isolation, a stable typed contract, and independent deployment/versioning that in-process function imports cannot. If tools are just Python functions in the agent's address space, then the agent process holds every credential and every capability those functions need — a buggy or jailbroken agent can reach anything importable. A protocol lets each capability live behind a boundary with exactly its own credentials and blast radius, lets you swap a server's implementation without touching the agent, and gives you a single choke point to log, rate-limit, and validate every call. The cost is serialization overhead and an extra hop, which is real but small relative to LLM latency, and well worth it when the capability is "execute SQL against a billed warehouse." **Why Helios demonstrates it.** Only `warehouse-mcp` holds a BigQuery client and the service-account credential; the agent process and the four stdio servers inherit no warehouse credentials, so a compromised Narrator literally cannot reach BigQuery — a guarantee in-process imports cannot make. **Follow-ups.** Where is the protocol overhead actually paid, and is it ever the bottleneck in a <5-min run? / How would you version a breaking change to `build_query`'s schema without a flag-day? / When would in-process tools genuinely be the right call?

Q3. stdio versus HTTP transport — how did you decide per server, and what are the tradeoffs?

Ideal answer. stdio runs the server as a child subprocess with JSON-RPC framed over stdin/stdout: lowest latency, no network surface, trivially co-located, but single-host and lifecycle-coupled to the parent. Streamable HTTP runs the server as a network service: it can live inside a network/security boundary, hold shared credentials, be reached by multiple clients, and scale independently — at the cost of a network surface that must be authenticated and a deployment to operate. The rule I apply: pure-compute, data-free, no-credential servers go on stdio; anything holding a sensitive credential or shared state that benefits from a security perimeter goes on HTTP. Transport is a deployment decision, not a capability one — the tool contract is identical either way. **Why Helios demonstrates it.** `semantic`, `stats`, `experiment`, and `report` run on stdio (pure compute, no warehouse credential), while `warehouse-mcp` runs over streamable HTTP precisely so it can sit in a network boundary with the BigQuery service account, be guarded by a bearer token (`HELIOS_WH_TOKEN`), and be shared across runs. **Follow-ups.** What would force you to move `stats-mcp` off stdio to HTTP? / How do you authenticate the HTTP transport, and what's the threat model? / How does transport choice interact with horizontal scaling of runs?

Q4. Why split capability across *five* servers instead of one big tool server?

Ideal answer. The split follows the trust boundaries, not arbitrary modularity. The single most important separation is SQL *generation* (semantic) from SQL *execution* (warehouse), so a query is validated and reconciled before a byte is scanned and so the model cannot smuggle hand-written SQL into the executor — they are different servers with different privileges. Math (stats) is separated from data fetch so statistical methods are version-pinned and unit-tested

independently of warehouse state. Experiment design and narration are isolated because they have zero data-access needs — they operate only on findings already produced. Each boundary is also a least-privilege and a blast-radius boundary: a bug or compromise is bounded by that server's tool catalog. The cost is more processes to operate; the payoff is that the trust story is *structural*. **Why Helios demonstrates it.** The five-server split lets the guardrail chain be physically real: `semantic-mcp` emits SQL strings but cannot execute them, `report-mcp` writes briefs but cannot query the warehouse, and only `warehouse-mcp` touches BigQuery — so "the model can't hand-author SQL" is enforced by which server holds which capability, not by a prompt. **Follow-ups.** Where would you draw the line if you had to collapse to three servers? / Doesn't five servers multiply operational overhead — how do you justify it for a portfolio-scale system? / Could one server's compromise still cascade, and how do you contain it?

Q5. Explain least privilege in this system — concretely, what can each server *not* do?

Ideal answer. Least privilege means every component holds exactly the credentials and scope it needs and nothing more, so the worst-case damage of a bug or compromise is bounded. In practice it shows up at two layers: per-server credential scoping and per-agent tool allow-lists. At the server layer, only the warehouse server has a BigQuery client, scoped to `bigquery.dataViewer` + `bigquery.jobUser`, read-only on the public dataset — no write, no delete, no other project. At the agent layer, each agent sees only the tools its role requires, so capability is sliced twice. The principle converts "please behave" into "you have no tool with which to misbehave." **Why Helios demonstrates it.** The per-agent allow-list is explicit: the Narrator is granted only `render_brief`, `save_diagnosis`, `export`, and `get_metric` — it has no `run_query` and no path to the warehouse — while the Decompose agent gets `build_query`, `run_query`, and `decompose_change` and nothing else, so even a misaligned agent's reach is bounded by its catalog. **Follow-ups.** Why scope at the agent layer when the server layer already isolates credentials? / What's the minimum privilege the Critic needs, and why does it differ from Diagnose? / How would you audit that the allow-lists match the actual call graph in production?

Q6. How is MCP different from "raw" function/tool calling, and when does that difference matter?

Ideal answer. Raw function calling is the model-side mechanic — the model emits a structured call against a schema you hand it inline, and your application code dispatches it. MCP is the system-side mechanic — a standardized way to *host, discover, isolate, and govern* the things being called across process and trust boundaries. They are complementary layers, not competitors: under the hood, an MCP client still surfaces tools to the model via the same structured tool-calling interface. The difference matters the moment tools need different credentials, independent deployment, cross-agent reuse, or a hard enforcement boundary — exactly when "a function in my process" stops being safe enough. For a single-agent script with no privilege separation, raw

tool calling is fine and MCP is overhead. **Why Helios demonstrates it.** Helios needs the difference: the same tools must be reused by seven agents with *different* allow-lists, the warehouse credential must be isolated from the agent process, and the SQL-generation/execution boundary must be enforced — none of which falls out of raw in-process function calling, all of which MCP makes structural. **Follow-ups.** If the SDK ultimately surfaces MCP tools as ordinary tool calls to the model, where exactly is the enforcement happening? / When is MCP overkill? / How does tool discovery via manifest help when the toolset evolves?

Q7. How does MCP make the grounding boundary *enforceable* rather than a prompt-time suggestion?

Ideal answer. A prompt that says "never write raw SQL" is advisory — a model can ignore it, and you only catch the violation after the fact, if at all. MCP makes it physical: you simply do not give the model a tool that executes arbitrary SQL. The only SQL-producing tool, `build_query`, interpolates *only* registry-defined metric templates and dimension expressions, so a hallucinated column like `event_params.foo` cannot survive composition — there is no template that references it, and unknown names raise a hard error. The model's reachable action space is the union of its allowed tools, so "can't" is a property of the wiring, not the wording. That is the difference between a policy and an invariant. **Why Helios demonstrates it.** `semantic-mcp` is the only path to SQL and `stats-mcp` is the only path to math; combined with `run_query` refusing any SQL not produced through that chain, Helios reaches its "0 hallucinated columns / 100% governed SQL" target by construction rather than by convention. **Follow-ups.** Could the model still hallucinate a *metric name* — and what happens when it does? / Where would a determined model find a gap in this boundary? / How do you prove to an auditor that 100% of executed SQL was governed?

Q8. What are MCP resources and prompts, and why did Helios lean on tools instead?

Ideal answer. Beyond tools, MCP defines *resources* (read-only addressable state the client can pull into context, like a file or a row set) and *prompts* (reusable parameterized instruction templates a server offers). Tools are for actions with side effects or computation; resources are for context the model should *read*; prompts standardize recurring instruction patterns. Helios is overwhelmingly action-oriented — compose a query, run it under budget, decompose, test significance, render — so tools dominate. Read-only context that could be a resource (the metric/dimension catalog, prior diagnoses) is instead surfaced through explicit tools (`list_dimensions`, `recall_prior`) so the access is logged in the audit trail and the model's reads are as governed as its actions. **Why Helios demonstrates it.** Even pure reads go through tools: `recall_prior` and `list_dimensions` are tool calls, not ambient resources, so every context fetch lands in the `audit_log` and the run stays fully reconstructable — a deliberate choice to keep auditability uniform across reads and writes. **Follow-ups.** When would exposing the semantic registry as a

resource beat exposing it via `get_metric` ? / Does routing reads through tools cost you anything in latency or tokens? / How would prompts help if you added a self-serve drill-down chat surface?

Q9. How do you keep the byte budget and reconciliation guarantees from being bypassable through MCP?

Ideal answer. A guarantee is only as strong as its narrowest enforcement point, so I put both checks inside the one server that can execute — they cannot be routed around because there is no other executor. Every `run_query` is hard-capped at the configured `max_bytes_billed`, and crucially `run_query` refuses any SQL whose normalized hash was not seen by a prior `dry_run` in the same run, so the cost estimate that gated the call is the cost that gets billed. Reconciliation lives there too: aggregates are checked against canonical totals via `reconcile`, and >0.5% drift fails the finding. Because the agents hold no other path to bytes, the budget is an invariant of the system, not a hope. **Why Helios demonstrates it.** `warehouse-mcp.run_query` raises `NotDryRunFirst` for any un-dry-run'd SQL and `ByteBudgetExceeded` past the fixed 5 GiB cap; with the agents' allow-lists granting no alternative executor, the per-run budget and the dry-run-before-run-query gate are unbypassable. **Follow-ups.** What stops an agent from issuing many small in-budget queries to blow the run total? / How does the same-run hash check survive query-cache or whitespace differences? / Where is the audit evidence that the budget was respected?

Tool Calling

Q10. What makes a good tool schema for an LLM, and how did you design Helios's?

Ideal answer. A good tool schema is narrow, typed, and self-documenting: it accepts only canonical, validatable inputs, returns a structured typed result (never prose the model must re-parse), and names its error modes so failures are recoverable rather than silent. I prefer enums and registry-bound names over free strings, explicit windows over implicit "recent," and outputs shaped so the *next* tool can consume them directly. The schema should make the wrong call hard to express — if a parameter can only be a known dimension, the model cannot pass garbage. Concretely, `build_query` takes `(metric | [metric], dims, filters, window)` and returns `{sql, governed: true, metrics, dims}`, so its output drops straight into `dry_run`. **Why Helios demonstrates it.** Helios's schemas are registry-bound and chainable: `build_query`'s typed output feeds `dry_run`, whose `{total_bytes_processed}` gates `run_query`, whose row output is summarized into `decompose_change`'s `t0 / t1` segment arrays — the schemas are designed as a pipeline, not isolated calls. **Follow-ups.** Why pass a window object instead of letting the model write the date predicate? / How do you keep output schemas stable as you add metrics? / What error fields does a tool need for the model to self-correct?

Q11. Walk through the guardrail chain `build_query` → `dry_run` → `run_query` . Why this exact order?

Ideal answer. The order encodes "govern, then price, then spend." `build_query` first guarantees the SQL is *legal* — composed only from registry metrics and known dimensions, so it can reference only real columns. `dry_run` then guarantees it is *affordable* — BigQuery returns the exact bytes it would scan without executing, and the server records the SQL's normalized hash. `run_query` only then *spends*, and only on SQL that passed both gates: it rejects anything not previously dry-run'd in the same run and caps `max_bytes_billed` . Reversing any step breaks a guarantee — you cannot price ungoverned SQL safely, and you cannot trust a cost estimate you then let the model mutate before execution. **Why Helios demonstrates it.** This is grounding rule G3 made physical in `warehouse-mcp` : the same-run hash check ties the executed query to the one whose cost was approved, so the chain is `agent` → `semantic.build_query` → `warehouse.dry_run` → `warehouse.run_query` , and only the warehouse server can complete it.

Follow-ups. What happens to the chain when `dry_run` reports over budget — retry, or something smarter? / Could the model dry-run a cheap query and run an expensive one? Why not? / Where does `reconcile` slot into this chain?

Q12. Why give the LLM *tools* instead of a database handle or a SQL string it can run?

Ideal answer. A database handle hands the model unbounded capability — arbitrary SQL, arbitrary cost, arbitrary tables, and an open door to hallucinated columns and runaway scans. Tools invert that: they expose a small set of safe, validated, budgeted operations, and the model composes *intent* (which metric, which dimensions, which window) rather than *implementation* (the SQL text). That preserves the three guarantees Helios sells — governed SQL, bounded cost, reconciled numbers — none of which can survive a raw handle. The model is good at deciding *what* to ask; it should never be trusted to author *how* against a billed, schema-fragile warehouse.

Why Helios demonstrates it. No Helios agent holds a database handle or a raw-SQL execution tool; data is reached only by composing governed metrics through `semantic-mcp` and executing through `warehouse-mcp` 's gated `run_query` , which is exactly why a hallucinated column is structurally impossible. **Follow-ups.** Doesn't forbidding raw SQL limit the questions the system can answer? How do you extend coverage? / What's your answer to "but text-to-SQL tools already do this"? / How would you handle a legitimately novel query the registry can't express yet?

Q13. Why do you insist that statistical tools be *deterministic*, and how do you guarantee it?

Ideal answer. Determinism means the same inputs always produce byte-identical outputs, which is non-negotiable for a system that must be reproducible, auditable, and regression-tested. If decomposition or significance were computed in token-space, the numbers would be non-reproducible, un-unit-testable, and silently wrong under paraphrase — you could never gate

them in CI. So all math runs in real, version-pinned `scipy/statsmodels/prophet` code behind `stats-mcp`, with a fixed RNG seed for any stochastic method; the model passes arrays in and reads numbers out, never computing. This lets me unit-test the decomposition against hand-worked golden values and assert the same answer on every run. **Why Helios demonstrates it.** `stats-mcp.decompose_change` implements the exact identity $\text{mix} = \sum \Delta w \cdot r(t_0)$, $\text{rate} = \sum w(t_0) \cdot \Delta r$, $\text{interaction} = \sum \Delta w \cdot \Delta r$, seeded at `rng_seed: 1729`, so the mix/rate/interaction split is byte-reproducible and is one of the keystone transforms with golden-value tests. **Follow-ups.** Which Helios statistics are actually stochastic, and how does the seed make them reproducible? / How would non-determinism here break your CI eval gate? / What's the failure mode if the model "rounds" a tool's number in the brief?

Q14. Sketch a tool-error taxonomy and the recovery strategy for each class.

Ideal answer. I group tool errors by the right *response*, not by where they originate. There are four useful classes: (1) **transient infrastructure** (warehouse/BigQuery flakiness, timeouts) — retry with bounded exponential backoff. (2) **budget/resource** (over-budget dry-run) — *do not* retry blindly; narrow the window or drop dimensions, then re-issue. (3) **governance/semantic** (unknown metric or dimension) — hard stop for that branch; re-plan against `list_dimensions`, never fall back to free SQL. (4) **data-quality/logic** (segment mismatch, zero sample, reconciliation drift) — fail the *finding*, not the run, and let the Critic record it. The discipline is that each class has one correct reaction, so the agent never papers over a governance error as if it were transient.

```
TRANSIENT (QueryTimeout, 5xx)      → retry x3, exponential backoff
BUDGET (ByteBudgetExceeded)        → narrow window/dims, re-dry_run (no blind retry)
GOVERNANCE(UnknownMetric/Dimension) → hard stop branch; re-plan via list_dimensions
DATA/LOGIC(SegmentMismatch, ZeroSample, reconcile >0.5%) → fail finding, log, prune
```

Why Helios demonstrates it. This is encoded in the agent layer: a `dry_run` budget rejection (G3) forces scope-narrowing rather than a blind retry, an unknown-name error (G5) is a hard branch stop, and a single failed Diagnose branch is logged and pruned rather than aborting the run, while a `>0.5%` reconcile drift fails just that finding. **Follow-ups.** Why is retrying an over-budget query specifically dangerous? / How do you stop a retry loop from blowing the 5-min/run SLA? / Which errors should surface to the brief versus stay in the audit log?

Q15. How do you prevent silent failures — a tool that returns plausible-but-wrong numbers?

Ideal answer. Silent failures are the dangerous ones because the model will confidently narrate them, so I defend with redundancy and reconciliation rather than trust. Every aggregate is reconciled against a canonical total (`reconcile`, `>0.5%` drift fails the finding), every finding is re-attacked by an independent adversarial agent, and every tool call — its args hash, the SQL run, bytes scanned, latency, verdict — is appended to an audit trail so a wrong number is traceable to

the exact call that produced it. Keystone transforms that fail silently (sessionization, the decomposition, revenue) get golden-value tests so a regression trips CI rather than shipping. The principle is verify-then-trust: a number earns its place in the brief by surviving an independent check. **Why Helios demonstrates it.** Revenue must reconcile to the cent (the `revenue_reconciles` test), `decompose_change` is golden-tested, and the Critic re-runs `significance_test` on holdout slices and probes for confounds before a finding can ship — three independent silent-failure tripwires. **Follow-ups.** Why is the decomposition specifically prone to silent failure? / How does an independent Critic catch what unit tests can't? / What's your "fail loud" path when reconciliation drifts?

Q16. How do you keep large tool outputs (row sets) from poisoning the context window?

Ideal answer. The model should see *aggregates and structure*, never raw row dumps — both to control cost and to prevent the model from re-deriving statistics it should be calling tools for. So tool outputs that could be large are summarized to the shape the next step needs before they enter the LLM's context: a query that returns thousands of rows is collapsed to per-segment (`weight`, `rate`) arrays for the decomposition, not streamed verbatim. Inter-stage hand-off is a compact typed JSON envelope, not prose, so token usage stays roughly flat even as the dimensional search widens. This keeps each agent's context small, cheap, and focused on decisions rather than data. **Why Helios demonstrates it.** Raw query result sets are summarized to aggregates before entering context — the LLM never sees row-level dumps — and stages hand off the typed Finding envelope (`finding_id`, `metric`, `dims`, `t0`, `t1`, `decomposition`, `significance`, `dollar_impact`, `evidence_query_ids`), so token usage stays flat as the hypothesis tree expands. **Follow-ups.** Where does the summarization happen — in the tool, the agent, or the control plane? / How do you keep `evidence_query_ids` enough to reconstruct without re-embedding the rows? / What breaks if a row set is summarized too aggressively?

Q17. How do you test tools in isolation from the unreliable LLM and the live warehouse?

Ideal answer. Tools are ordinary typed functions, so they get ordinary deterministic tests — independent of the model and, where possible, of the warehouse. Pure-compute servers (stats, experiment) are unit-tested against hand-worked golden values with a fixed seed, with zero warehouse dependency. The semantic server is tested by asserting that `build_query` emits the expected governed SQL for known metric/dimension combinations and rejects unknown names, without executing anything. Warehouse interactions are validated via `dry_run` (no spend) and golden-row fixtures, and the full pipeline is graded end-to-end by the offline eval harness on a frozen dataset — so a tool regression is caught at the unit level *and* the system level before it can reach a real run. **Why Helios demonstrates it.** `stats-mcp` runs entirely on stdio with no credentials precisely so it is unit-testable in isolation, and the keystones — `decompose_change`, sessionization, revenue reconciliation — carry golden-value tests, while the CI eval gate grades

the assembled pipeline on 50 frozen labeled scenarios. **Follow-ups.** How do you write a golden test for a stochastic forecast? / How do you test the *boundary* — that `run_query` truly refuses un-dry-run'd SQL? / What's the cheapest test that would have caught a hallucinated-column regression?

Agent Orchestration

Q18. Why a deterministic finite-state machine with model-driven *nodes*, rather than letting the LLM decide the flow?

Ideal answer. Control flow and reasoning are different jobs and should be owned by different things. Letting an LLM decide *what to do next* (an open agent loop) is non-reproducible, hard to budget, and hard to test — the same input can take different paths on different runs. So I make the *flow* deterministic code — a finite-state machine with named states and explicit transitions — while the LLM is a *composer within a node*: at each state it decides which tool calls to emit, but it cannot change the order of stages or skip the Critic. This gives the reasoning flexibility of agents (a best-first hypothesis search, an adversarial refutation) with the reproducibility, auditability, and cost control of a fixed pipeline. The slogan: the model composes tool calls; it never controls flow.

Why Helios demonstrates it. The Orchestrator drives a fixed FSM — `PLAN → MONITOR → (CLEAN_RUN | DECOMPOSE → DIAGNOSE → CRITIC LOOP → PRESCRIBE → NARRATE) → END` — and worker output never ships directly; it must pass the Critic loop, so the diagnosis path is identical and reconstructable every run. **Follow-ups.** What concretely becomes untestable if the LLM controls transitions? / How do you handle a genuinely novel case the FSM has no state for? / Where in a node does the model still have real latitude?

Q19. Why seven agents instead of one well-prompted model?

Ideal answer. Specialization and adversarial review are the two things a single prompt cannot do well. A monolithic prompt asked to both confidently propose a root cause *and* rigorously refute it is in conflict with itself — confirmation bias is structural. Splitting into role-specialized agents lets each have a tight system prompt, a minimal tool allow-list, and an isolated, testable responsibility, and lets one agent be purpose-built to *attack* another's output. It also lets me right-size the model per role and keep each agent's context small. The cost is orchestration complexity and inter-agent latency, which the deterministic FSM and typed hand-offs keep bounded. Multi-agent here is not fashion; it is how I get an adversarial check and least-privilege at the same time. **Why Helios demonstrates it.** The seven agents (Orchestrator, Monitor, Decompose, Diagnose, Prescribe, Narrator, Critic) each own one stage with its own allow-list, and the Critic exists specifically to refute the Diagnose agent — a separation a single prompt cannot honestly reproduce. **Follow-ups.** Where is the latency cost of seven hops, and is it inside the 5-min

budget? / Which two agents could you merge with least harm, and which must never merge? / How do you stop the agents from duplicating each other's queries?

Q20. How did you decide Opus versus Sonnet per agent?

Ideal answer. I matched model capability to the reasoning depth and cost-of-error of each role, not to seniority for its own sake. Opus goes to the three roles that search an open hypothesis space or carry a high cost of being wrong: the Orchestrator (global planning, budget allocation, branch routing), Diagnose (combinatorial hypothesis-tree search with self-directed SQL verification), and the Critic (adversarial reasoning that must out-think Diagnose to find a confound). Sonnet handles the high-volume, bounded-action-space stages — Monitor, Decompose, Prescribe, Narrator — where the decision is "which series to test," "which decomposition to run," "which power analysis to call," "which template to fill." This keeps cost per run inside the fixed budget while spending the expensive model only where judgment actually moves accuracy. **Why Helios demonstrates it.** The Opus/Sonnet split is exactly Orchestrator/Diagnose/Critic on Opus and Monitor/Decompose/Prescribe/Narrator on Sonnet — reserving the costly model for open-ended reasoning and adversarial critique, the two places error is most expensive. **Follow-ups.** Could the Decompose agent be Sonnet *because* the math is in a deterministic tool — is that the reason? / When would you promote Monitor to Opus? / How do you measure whether an Opus role is worth its cost?

Q21. Explain the Critic loop — verdicts, the retry bound, and why an adversarial agent matters.

Ideal answer. The Critic is a dedicated adversary that tries to *refute* every candidate finding along the four classic failure modes — mix confound, insufficient sample, seasonality, and data-quality artifact — and returns one of three verdicts. PASS: the finding survived refutation and proceeds to Prescribe. DOWNGRADE: plausible but under-evidenced (sample too small, decomposition not isolated to one dimension), so it returns to Diagnose for a *targeted* re-query, bounded by `MAX_REFUTE_ROUNDS = 2`; after the cap it is demoted to a watchlist note rather than a confident finding. DROP: refuted (confound found, fails reconciliation, fully explained by known seasonality/launch from memory), discarded and locally suppressed so siblings don't re-raise it. The bound matters because an unbounded refute loop would blow the time and byte budgets; the adversary matters because the cheapest way to make a finding trustworthy is to try hard to break it first. **Why Helios demonstrates it.** The Critic (Opus) consults `recall_prior` for seasonality/launch and suppression, re-runs `significance_test` on holdout slices, and probes for a traffic-mix confound via `run_query` — a structurally separate refutation pass that is the third defensibility pillar alongside governed SQL and deterministic stats. **Follow-ups.** Why cap at two refute rounds specifically — what happens at the cap? / How does the Critic avoid simply agreeing with Diagnose? / What does a DROP add to suppression, and how is that scoped to the run?

Q22. What is the Finding envelope, why typed JSON, and how does it enable hand-off?

Ideal answer. The Finding envelope is the single typed contract that every stage produces and consumes, so agents communicate via structured data, not prose. It carries the identity and full evidentiary state of a candidate: the metric, dimension slice, time window, the mix/rate/interaction decomposition, the significance result, the dollar revenue-at-risk, and pointers to the evidence queries. Typed JSON matters for three reasons: it is validatable (a malformed finding fails fast), it is compact (token usage stays flat as the search widens), and it is the same shape the memory store persists — so a finding flows from Diagnose through Critic to Narrator to `save_diagnosis` without re-parsing. Prose hand-off would be lossy, unvalidatable, and would tempt downstream agents to re-derive numbers they should inherit. **Why Helios demonstrates it.** The hand-off envelope is exactly `{finding_id, metric, dims, t0, t1, decomposition, significance, dollar_impact, evidence_query_ids}`, and it maps onto the `diagnosis_history` schema so the same typed object the Critic vets is what the Narrator renders and `save_diagnosis` persists.

Follow-ups. Why include `evidence_query_ids` rather than the rows themselves? / What validation runs on the envelope between stages? / How does the envelope's shape line up with what memory needs to recall later?

Q23. How does context compaction work across the pipeline, and why is it load-bearing?

Ideal answer. Compaction means each agent receives the *minimum* context it needs to act, summarized and structured, rather than the accumulated transcript of the whole run. Concretely a worker gets: the run plan, the canonical metric/dimension catalog, the upstream stage's typed output (JSON, not prose), and any priors from `recall_prior` — and raw result sets are reduced to aggregates before they enter context. This is load-bearing for two reasons: cost and time (a flat, small context per agent keeps a multi-agent run inside the <5-min and token budgets), and *quality* (a focused context reduces distraction and keeps the model on its one job). Without compaction, a deep hypothesis-tree search would balloon context linearly with the number of branches and both cost and latency would blow. **Why Helios demonstrates it.** Helios compacts deliberately — summarized aggregates instead of row dumps, typed Finding envelopes instead of prose, and `recall_prior` priors decayed by `exp(-age_days/HALF_LIFE)` — so token usage stays flat even as Diagnose widens its dimensional search to the bounded `MAX_BRANCHING = 4`, `MAX_DEPTH = 3`. **Follow-ups.** What information is safe to drop in compaction, and what must always survive? / How does memory decay interact with context size over many runs? / How would you detect that compaction silently dropped a needed fact?

Q24. How does the FSM handle failure isolation, retries, and the clean-run case?

Ideal answer. A robust pipeline degrades gracefully and never ships a partial result. Failure is isolated by scope: a single failed Diagnose branch is logged and pruned, not fatal, because one

dead hypothesis shouldn't kill the run; but if Monitor or Decompose fail entirely, the Orchestrator aborts and writes an audit record rather than emitting a half-built brief. Retries are class-specific: transient warehouse errors back off and retry three times, a budget rejection forces scope-narrowing instead of a blind retry, and a governance error hard-stops that branch. The clean-run case is first-class: if Monitor finds no anomaly, the FSM short-circuits to a "no material change" brief and *still* writes a run-state audit row, so absence-of-finding is itself recorded and the run is reproducible. **Why Helios demonstrates it.** The FSM's `CLEAN_RUN → NARRATE(no-finding brief) → END` branch and the abort-with-audit-on-Monitor/Decompose-failure path are explicit, and every tool call lands in `audit_log` (args hash, SQL, bytes, verdict) so even a clean or aborted run is fully reconstructable. **Follow-ups.** Why record a clean run at all? / How do you prevent a flaky branch from silently degrading accuracy? / What's the recovery if the Orchestrator itself crashes mid-run?

Q25. How do you keep an autonomous multi-agent run cheap, bounded, and on-time?

Ideal answer. Autonomy is only safe if it is bounded, so cost and time are enforced at every layer rather than hoped for. The byte budget is hard-capped per `run_query` and the dry-run gate prices every query before it spends; the hypothesis search is breadth- and depth-bounded so it cannot fan out indefinitely; the Critic loop is capped at two refute rounds; the model split spends Opus only on reasoning-critical roles; and context compaction keeps token cost flat. The Orchestrator also watches the running byte total and tightens scope mid-run if it nears the cap. Together these turn "an AI agent loose on a warehouse" into a run with a known worst-case cost and a sub-five-minute SLA — which is what makes scheduling it unattended defensible. **Why Helios demonstrates it.** The fixed 5 GiB byte budget (`byte_budget: 5368709120`), `MAX_BRANCHING = 4` / `MAX_DEPTH = 3` in Diagnose, `MAX_REFUTE_ROUNDS = 2` in the Critic, the Opus/Sonnet split, and mid-run scope-tightening off the `run_state.bytes_scanned` counter jointly hold each run under 5 minutes and inside budget. **Follow-ups.** Which bound binds first in practice — bytes, time, or tokens? / How do you choose the depth/branching caps without hurting the 85% accuracy target? / What's your alerting if a scheduled run breaches its budget?